Arab American University

Faculty of Engineering

Computer Systems Engineering

Parallel & Distributed Systems

Project 2

Name: Feras jarrar ID: 202111567

Supervised by: Dr. Hussein Younis

Introduction

The algorithm I have chosen is Merge Sort which is an algorithm that sorts arrays using divide and conquer the reason I have chosen this algorithm because it is easy to parallelize since the array can be divided between different pthreads.

Sequential implementation

This function will recursively call itself dividing the array by half each time and each half will be divided till there is only 1 element which is considered to be sorted then it will be sent to the merge function to sort these elements in the array.

```
void merge_sort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        merge_sort(arr, l, m);
        merge_sort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
```

This function only receives sorted parts of the array when called from it arguments and sort the parts by comparing each element and putting the smaller element first in the array until all the elements are sorted then when one of the 2 arrays is all sorted it will put the rest of the elements of the other array after the sorted elements since all the rest of the elements are sorted as well.

```
void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];

    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

Time Methodology

```
int main() {
    float time[5];
    for(int i=0; i<5; i++){
    int arr[SIZE];

    for(int i=0; i < SIZE; i++) {
        arr[i] = rand() % 10001;
    }

    auto start = std::chrono::high_resolution_clock::now();
    merge_sort(arr, 0, SIZE - 1);
    auto end = std::chrono::high_resolution_clock::now();
    check_sort(arr);
    printf("\nTime taken: %f seconds\n", std::chrono::duration<double>(end - start).count());
    time[i] = std::chrono::duration<double>(end - start).count();
}
    float avg = (time[0] + time[1] + time[2] + time[3] + time[4]) / 5.0;
    printf("\nAverage time taken: %f seconds\n", avg);
    return 0;
}
```

This is the main section of my code and is the start and end of the main thread the first two line of codes would create a for loop which will sort the function 5 times and get the time for each one to calculate the average time to calculate the time I used the chrono library putting the start before calling the merge sort function and the end after calling the merge sort to accurately calculate the time and putting the total time (end-start) into an array each time after the loop the average time be printed with each cycle time and it success.

Parallelization strategy

I parallelized the Algorithm by dividing the array between the threads number so each thread will have different array locations and each sort it half after that the main thread will wait until all threads are finished to merge each thread part.

Experiments

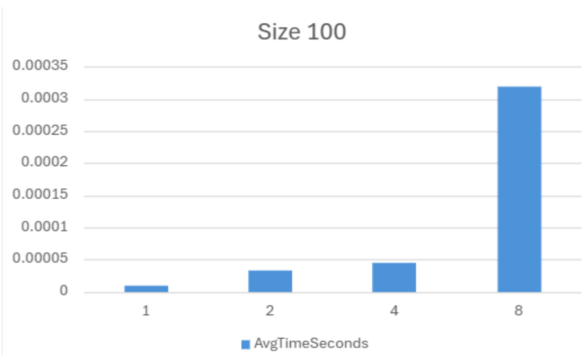
Hardware specs: 4 cores, 8 logical cores, windows 11 Home, core i5 9300h.

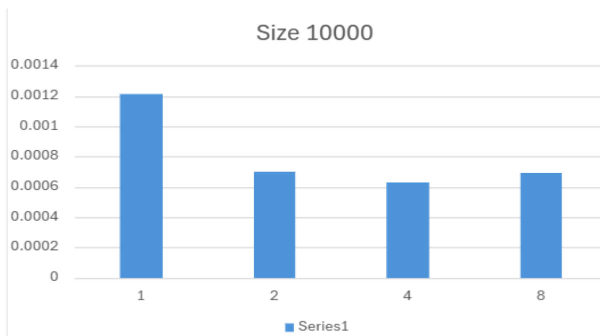
Input sizes and thread counts

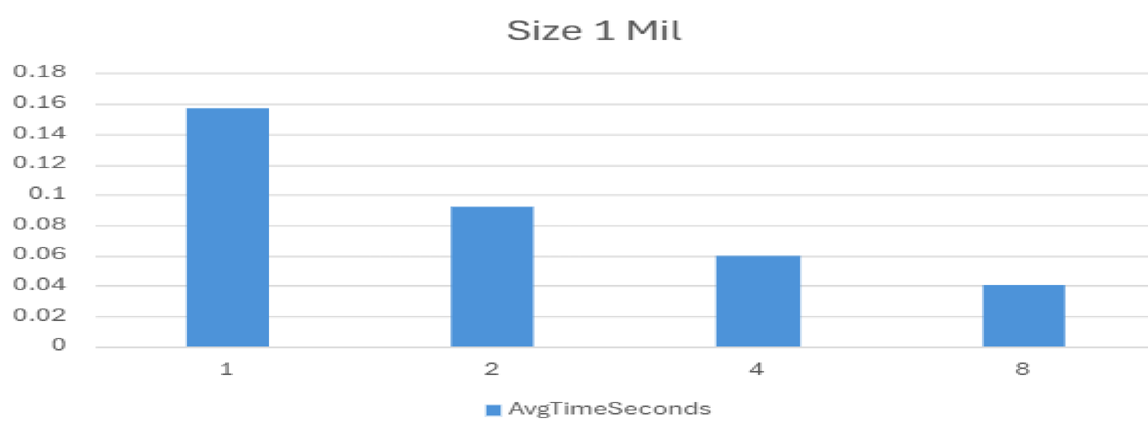
- Array sizes: 100, 10000, 1000000 (1 mil)
- Thread counts: 1, 2, 4 ,8

Results

Graphs:







Size	Sequential	2Threads	4Threads	8Threads
100	0.000008	0.000032	0.000044	0.00032
10000	0.001295	0.0007	0.00063	0.00069
1 Mil	0.159338	0.092	0.06	0.04

Speed up Table:

Thread num	100	10000	1 Mil
2 Threads	0.25x	1.85x	1.74x
4 Threads	0.18x	2.06x	2.67x
8 Threads	0.025x	1.88x	4.00x

Discussion

For the 8 threads the speed up was pretty good at large arrays because the size of the parallel size of the code was way larger than the sequential part of the code however it can be noted that the load for each core was different due to the number of threads and array size, and the speed was slightly different each time due to core use sometimes a core would be used for the os which would result in slower time.

Conclusion

Transforming an algorithm from sequential to parallel can be challenging due to the different methods available the Synchronization and finding and observing critical sections of the code however in most cases parallelizing an algorithm won't be worth the effort in most case if the input size was small and might even return diminishing results also parallelizing an algorithm may result in special cases which also needs to be observed.

Note: The algorithm in omp is faster than the pthread one due to the implementation difference for pthread I had to interfere with the implementation and I messed it up which resulted in a way higher time for the algorithm in smaller inputs while in the omp I didn't have much interference with the implementation of the threads due to it nature which had much better results.